

Регулярні вирази

Розділ 11

Python для всіх
www.py4e.com



Регулярні вирази

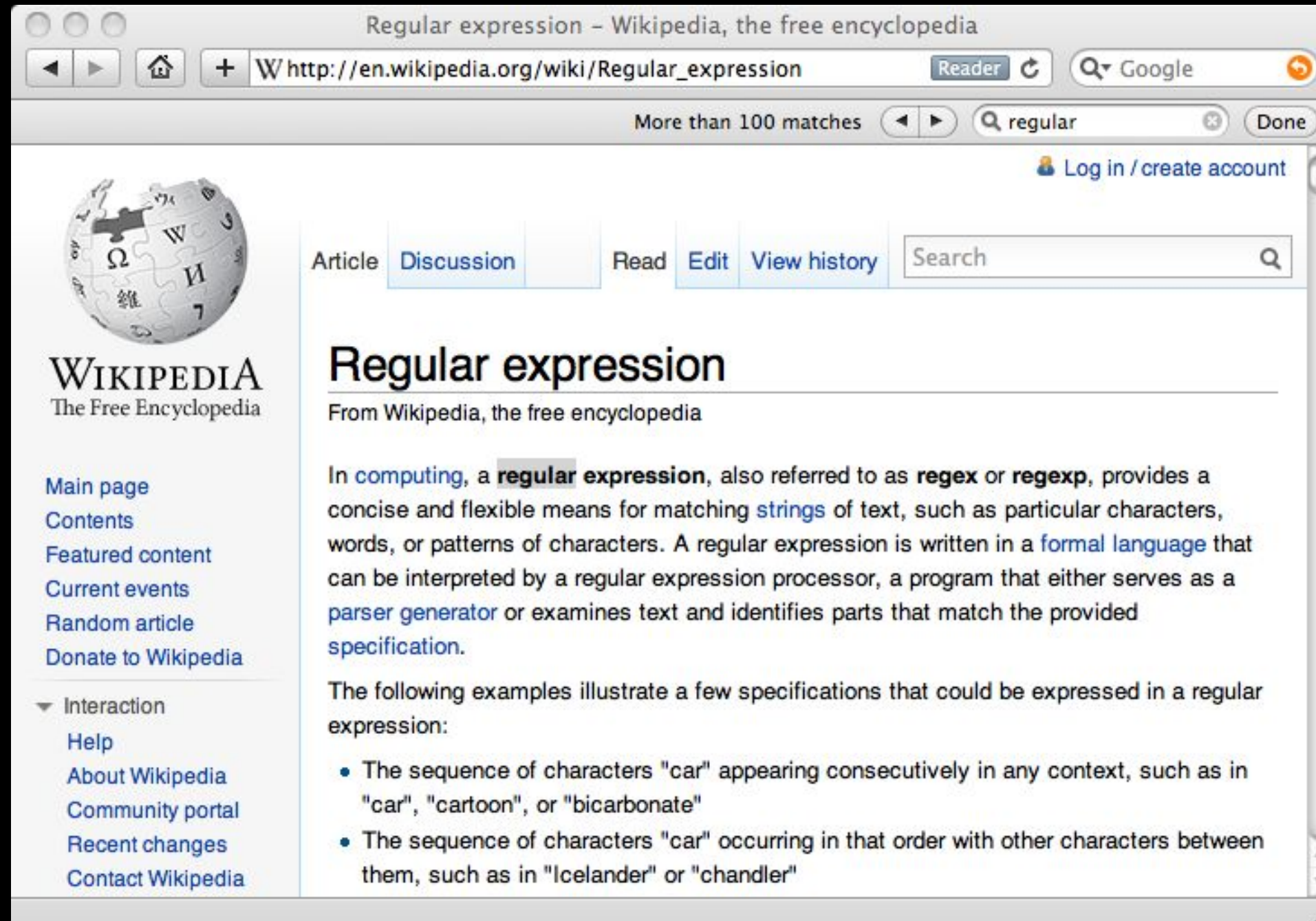
У комп'ютерних технологіях регулярний вираз, який також називають «regex» або «regexr», забезпечує стислий і гнучкий спосіб для пошуку фрагментів тексту, таких як окремі символи, слова або шаблони символів. Регулярний вираз записується формальною мовою, яка може бути інтерпретована процесором регулярних виразів.

http://en.wikipedia.org/wiki/Regular_expression

Регулярні вирази

Дійсно розумні вирази із «байдужими символами» (замінними знаками) для пошуку, виокремлення та парсингу рядків

http://en.wikipedia.org/wiki/Regular_expression



Дійсно розумне використання «Find» або «Search»

Що варто знати про регулярні вирази?

- Дуже потужні та досить загадкові.
- Класні, якщо їх дійсно зрозуміти.
- Регулярні вирази – це мова всередині мови.
- Мова «символів-маркерів» – програмування символами.
- Це свого роду «олдскульна» мова – вона компактна.

WHENEVER I LEARN A NEW SKILL I CONCOCT ELABORATE FANTASY SCENARIOS WHERE IT LETS ME SAVE THE DAY.

OH NO! THE KILLER MUST HAVE FOLLOWED HER ON VACATION!



BUT TO FIND THEM WE'D HAVE TO SEARCH THROUGH 200 MB OF EMAILS LOOKING FOR SOMETHING FORMATTED LIKE AN ADDRESS!

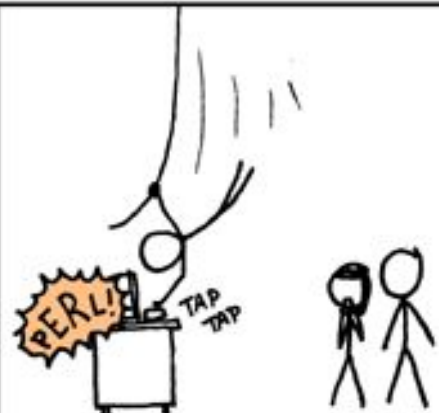


IT'S HOPELESS!

EVERYBODY STAND BACK.



I KNOW REGULAR EXPRESSIONS.



<http://xkcd.com/208/>

Короткий посібник з регулярних виразів

<code>^</code>	Позначає початок рядка
<code>\$</code>	Позначає кінець рядка
<code>.</code>	Позначає будь-який символ
<code>\s</code>	Позначає пробіл
<code>\S</code>	Позначає будь-який символ, який не є пробілом
<code>*</code>	Повторює символ нуль або кілька разів
<code>*?</code>	Повторює символ нуль або кілька разів (не жадібно)
<code>+</code>	Повторює символ один або кілька разів
<code>+?</code>	Повторює символ один або кілька разів (не жадібно)
<code>[aeiou]</code>	Відповідає одному символу з набору у списку
<code>[^XYZ]</code>	Відповідає одному символу не з набору у списку
<code>[a-z0-9]</code>	Набір символів може містити діапазон
<code>(</code>	Вказує, з якого місця слід почати витягування з рядка
<code>)</code>	Вказує, у якому місці слід закінчити витягування з рядка

Модуль регулярних виразів

- Перш ніж ви зможете використовувати регулярні вирази у вашій програмі, ви повинні імпортувати бібліотеку за допомогою «`import re`».
- Можна використати `re.search()` для перевірки відповідності рядка регулярному виразу, подібно до використання методу `find()` для рядків.
- Можна використати `re.findall()` для витягування частин рядка, які відповідають вашому регулярному виразу, подібно до комбінації `find()` та використання зрізу: `var[5:10]`.

Використання `re.search()` як `find()`

```
hand = open('mbox-short.txt')
for line in hand:
    line = line.rstrip()
    if line.find('From:') >= 0:
        print(line)
```

```
import re

hand = open('mbox-short.txt')
for line in hand:
    line = line.rstrip()
    if re.search('From:', line) :
        print(line)
```

Використання `re.search()` як `startswith()`

```
hand = open('mbox-short.txt')
for line in hand:
    line = line.rstrip()
    if line.startswith('From:') :
        print(line)
```

```
import re

hand = open('mbox-short.txt')
for line in hand:
    line = line.rstrip()
    if re.search('^From:', line) :
        print(line)
```

Ми налаштовуємо те, що повертає пошук, додаючи до рядка спеціальні символи

«Байдужі» символи – замінні знаки

- Символ **крапка** відповідає будь-якому символу.
- Якщо ви додасте символ **зірочка**, символ буде повторюватись «будь-яку кількість разів»

X-Sieve: CMU Sieve 2.3
X-DSPAM-Result: Innocent
X-DSPAM-Confidence: 0.8475
X-Content-Type-Message-Body: text/plain

Позначає початок
рядка

Багато
разів

Позначає будь-який
СИМВОЛ

X . * :

Налаштування результатів пошуку

Залежно від того, наскільки «чистими» є ваші дані та яка мета вашого пошуку, ви можете трохи звузити знайдені результати.

X-Sieve: CMU Sieve 2.3

X-DSPAM-Result: Innocent

X-Plane is behind schedule: two weeks

X-: Very short

Позначає
початок рядка

Багато
разів

Позначає будь-який
СИМВОЛ

X.*:

Налаштування результатів пошуку

Залежно від того, наскільки «чистими» є ваші дані та яка мета вашого пошуку, ви можете трохи звузити знайдені результати.

X-Sieve: CMU Sieve 2.3

X-DSPAM-Result: Innocent

X-: Very Short

X-Plane is behind schedule: two weeks

Позначає початок
рядка

Один або
кілька разів

X- \ S+ :

Позначає символи, які не є пробілами

Пошук та витягування даних

- `re.search()` повертає правдиво / хибно залежно від того, чи знайдено рядок за допомогою регулярного виразу.
- Якщо ми дійсно хочемо витягти відповідні рядки, ми використовуємо `re.findall()`

`[0-9]+`



Одна або кілька
цифр

```
>>> import re
>>> x = 'My 2 favorite numbers are 19 and 42'
>>> y = re.findall('[0-9]+', x)
>>> print(y)
['2', '19', '42']
```


Пошук та витягування даних

Коли ми використовуємо `re.findall()`, повертається список нулем або більшою кількістю підрядків, які відповідають регулярному виразу.

```
>>> import re
>>> x = 'My 2 favorite numbers are 19 and 42'
>>> y = re.findall('[0-9]+', x)
>>> print(y)
['2', '19', '42']
>>> y = re.findall('[AEIOU]+', x)
>>> print(y)
[]
```


Застереження: Жадібний пошук

Повторювані символи (* та +) виштовхуються **назовні** в обох напрямках (жадібно), щоб знайти якомога довший рядок.

Один або більше символів

```
>>> import re
>>> x = 'From: Using the : character'
>>> y = re.findall('^F.+:', x)
>>> print(y)
['From: Using the :']
```

^ F . + :

Чому не «From:» ?

F - перший символ у
результатах пошуку

: - останній символ у
результатах пошуку

Нежадібний пошук

Не всі коди у регулярних виразах жадібні! Якщо ви додасте символ **?**, то **+** і ***** трохи розслабляться...

```
>>> import re
>>> x = 'From: Using the : character'
>>> y = re.findall('^F.+?:', x)
>>> print(y)
['From: ']
```

Один або
більше
символів
нежадібно

The diagram shows the regular expression pattern `^F.+?:` with several annotations. A green arrow points to the `F`, a yellow arrow points to the `:`, and an orange arrow points to the `+`. The `.` and `?` are also present in the pattern.

F - перший символ у
результатах пошуку

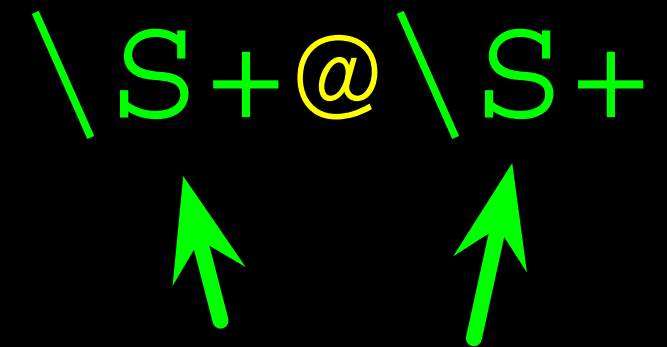
: - останній символ у
результатах пошуку

Налаштування витягування рядків

Ви можете уточнити пошук для `re.findall()` і окремо визначити, яку частину пошуку буде витягнуто, використовуючи круглі дужки

From `stephen.marquard@uct.ac.za` Sat Jan 5 09:14:16 2008

```
>>> y = re.findall('\S+@\S+', x)
>>> print(y)
['stephen.marquard@uct.ac.za']
```



The diagram shows the regular expression pattern `\S+@\S+` in blue text. Two red arrows point upwards from the text below to the `\S+` parts of the pattern, highlighting that any non-whitespace character is acceptable.

Хоча б один
СИМВОЛ, який
не є пробілом

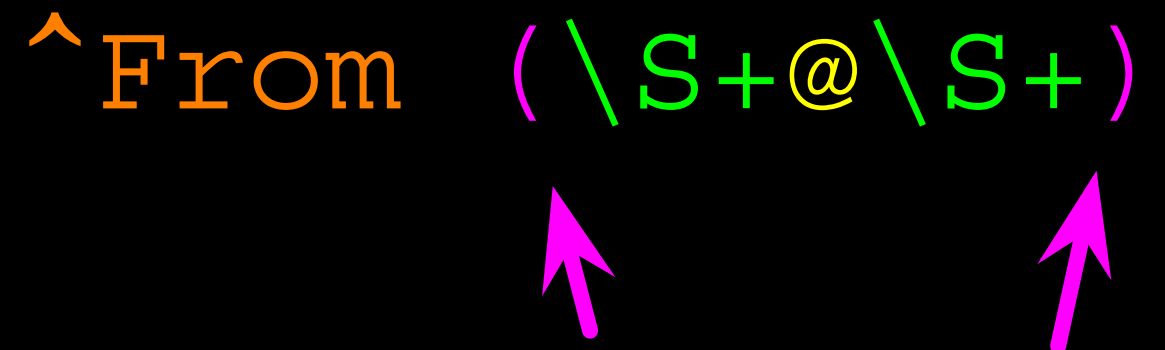
Налаштування витягування рядків

Дужки не є частиною пошуку, але вони вказують, де початок та завершення у витягуванні рядка

From stephen.marquard@uct.ac.za Sat Jan 5 09:14:16 2008

```
>>> y = re.findall('\S+@\S+', x)
>>> print(y)
['stephen.marquard@uct.ac.za']
>>> y = re.findall('^From (\S+@\S+)', x)
>>> print(y)
['stephen.marquard@uct.ac.za']
```

^From (\S+@\S+)



The diagram illustrates the regex pattern `^From (\S+@\S+)`. The word `From` is in orange, and the parentheses and their contents are in green. Two blue arrows point to the opening and closing parentheses, indicating that the text between them is the captured group.

Приклади парсингу рядків...

21 31
↓ ↓
From stephen.marquard@uct.ac.za Sat Jan 5 09:14:16 2008

```
>>> data = 'From stephen.marquard@uct.ac.za Sat Jan 5 09:14:16 2008'
>>> atpos = data.find('@')
>>> print(atpos)
21
>>> sppos = data.find(' ', atpos)
>>> print(sppos)
31
>>> host = data[atpos+1 : sppos]
>>> print(host)
uct.ac.za
```

Витягування імені хоста
за допомогою find та
використання зрізу
рядка

Шаблон подвійного розбиття

Іноді ми розбиваємо рядок, а потім беремо один з відрізків і розбиваємо його знову

```
From stephen.marquard@uct.ac.za Sat Jan 5 09:14:16 2008
```

```
words = line.split()  
email = words[1]  
pieces = email.split('@')  
print(pieces[1])
```

```
stephen.marquard@uct.ac.za  
['stephen.marquard', 'uct.ac.za']  
'uct.ac.za'
```

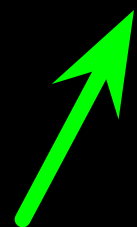
Шаблон з Regex

From stephen.marquard@uct.ac.za Sat Jan 5 09:14:16 2008

```
import re
lin = 'From stephen.marquard@uct.ac.za Sat Jan 5 09:14:16 2008'
y = re.findall('@([^\s]*)', lin)
print(y)
```

['uct.ac.za']

'@([^\s]*)'



Проходити рядком, поки не знайдеться знак @

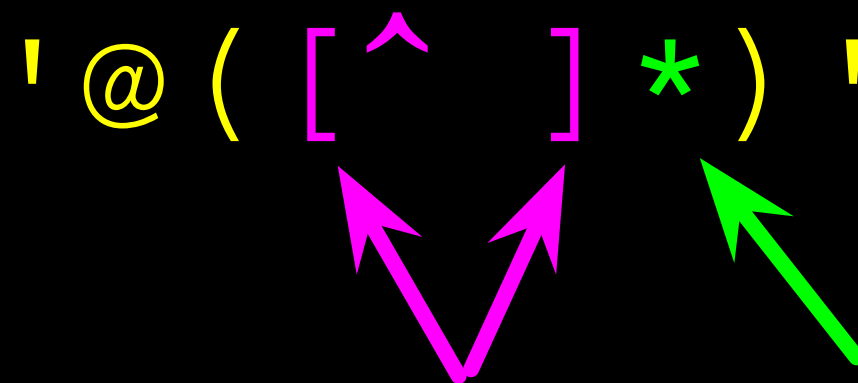
Версія з регулярними виразами

```
From stephen.marquard@uct.ac.za Sat Jan 5 09:14:16 2008
```

```
import re  
lin = 'From stephen.marquard@uct.ac.za Sat Jan 5 09:14:16 2008'  
y = re.findall('@([ ^ ]*)', lin)  
print(y)
```

```
['uct.ac.za']
```

' @ ([^] *) '



Пошук непорожніх символів

Пошук багатьох

Версія з регулярними виразами

```
From stephen.marquard@uct.ac.za Sat Jan 5 09:14:16 2008
```

```
import re  
lin = 'From stephen.marquard@uct.ac.za Sat Jan 5 09:14:16 2008'  
y = re.findall('@([ ^ ]*)', lin)  
print(y)
```

```
['uct.ac.za']
```

'@([^]*)'

Витягування непорожніх символів

Ще краща версія з регулярними виразами

```
From stephen.marquard@uct.ac.za Sat Jan  5 09:14:16 2008
```

```
import re  
lin = 'From stephen.marquard@uct.ac.za Sat Jan  5 09:14:16 2008'  
y = re.findall('^From .*@([ ^]*)', lin)  
print(y)
```

```
['uct.ac.za']
```

'^From .*@([^]*)'

A diagram illustrating the components of the regular expression pattern. A green arrow points from the text 'Почати з початку рядка' (Start from the beginning of the line) to the '^' character in the pattern. A purple arrow points from the text 'шукати рядок 'From'' (search for the line 'From') to the word 'From' in the pattern.

Почати з початку рядка, шукати рядок 'From'

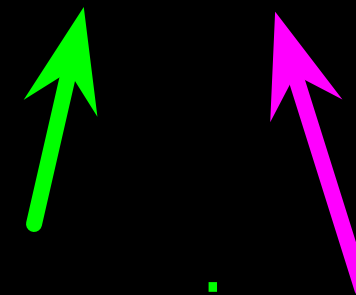
Ще краща версія з регулярними виразами

```
From stephen.marquard@uct.ac.za Sat Jan 5 09:14:16 2008
```

```
import re  
lin = 'From stephen.marquard@uct.ac.za Sat Jan 5 09:14:16 2008'  
y = re.findall('^From .*@([ ^]*)', lin)  
print(y)
```

```
['uct.ac.za']
```

'^From . * @ ([^] *) '



Пропустити купу символів, шукати символ @

Ще краща версія з регулярними виразами

```
From stephen.marquard@uct.ac.za Sat Jan 5 09:14:16 2008
```

```
import re  
lin = 'From stephen.marquard@uct.ac.za Sat Jan 5 09:14:16 2008'  
y = re.findall('^From .*@([ ^]*)', lin)  
print(y)
```

```
['uct.ac.za']
```

'^From .*@([^]*)'



Початок витягування

Ще краща версія з регулярними виразами

From stephen.marquard@uct.ac.za Sat Jan 5 09:14:16 2008

```
import re
lin = 'From stephen.marquard@uct.ac.za Sat Jan 5 09:14:16 2008'
y = re.findall('^From .*@([ ^ ]*)', lin)
print(y)
```

['uct.ac.za']

'^From .*@([^]+)'

Пошук непорожніх символів Пошук багатьох з них

Ще краща версія з регулярними виразами

```
From stephen.marquard@uct.ac.za Sat Jan 5 09:14:16 2008
```

```
import re  
lin = 'From stephen.marquard@uct.ac.za Sat Jan 5 09:14:16 2008'  
y = re.findall('^From .*@([ ^]*)', lin)  
print(y)
```

```
['uct.ac.za']
```

```
'^From .*@([ ^]+)'
```



Завершення витягування

Пошук спаму

```
import re
hand = open('mbox-short.txt')
numlist = list()
for line in hand:
    line = line.rstrip()
    stuff = re.findall('^X-DSPAM-Confidence: ([0-9.]+)', line)
    if len(stuff) != 1 : continue
    num = float(stuff[0])
    numlist.append(num)
print('Maximum:', max(numlist))
```

X-DSPAM-Confidence: 0.8475

python ds.py

Maximum: 0.9907

Екранований символ

Якщо ви хочете, щоб спеціальний символ регулярного виразу поведився **нормально** (у більшості випадків), перед ним потрібно поставити `\`.

```
>>> import re
>>> x = 'We just received $10.00 for cookies.'
>>> y = re.findall('\$[0-9.]+', x)
>>> print(y)
['$10.00']
```

Щонайменше
один або більше

\\$ [0-9.] +

Справжній знак
долара

Цифра або крапка

Підсумки

- Регулярні вирази – це загадкова, але потужна мова для пошуку рядків і витягування фрагментів з цих рядків.
- Регулярні вирази мають спеціальні символи, які вказують на наміри.

Права власності / Застереження



Авторські права на ці слайди з 2010 року належать Чарльзу Северенсу (www.dr-chuck.com) зі Школи інформації Мічиганського університету та захищені ліцензією Creative Commons Attribution 4.0. Будь ласка, збережіть цей фінальний слайд у всіх копіях документа, щоб відповідати вимогам ліцензії щодо посилань на джерела. При повторній публікації матеріалів, якщо щось зміните, додайте ім'я та організацію до переліку співавторів нижче.

Першоджерело: Чарльз Северенс, Школа інформації Мічиганського університету

Переклад: платформа Prometheus